



LECTURE 15: BINARY SEARCH — SOLVING 3 PROBLEMS

Topic: Binary Search on Structure & Answer Space

“ Binary Search ka matlab fast code likhna nahi hai.

Binary Search ka matlab hai **galat possibilities ko jaldi aur safely hata dena.** ”



WHY THIS LECTURE MATTERS (BIG PICTURE)

Real world me:

- data bada hota ja raha hai
- brute force kaam nahi karta
- har decision ka **cost** hota hai

Binary Search sikhat hai:

- kaise **minimum steps me maximum clarity** laayein
- kaise **guarantee ke saath decision** lein

Companies DSA isliye puchti hain kyunki:

“Tum pressure me kaise sochte ho?”



UNIVERSAL BINARY SEARCH CHECKLIST (LOCK THIS)

Kisi bhi problem par binary search lagane se pehle ye 3 sawal poocho:

- 1 Kya search space **ordered / monotonic** hai?
- 2 Kya mid par khade ho kar **clear decision** le sakta hoon?
- 3 Kya decision ke baad **aadha space discard** ho raha hai?

Agar teeno YES → **Binary Search confirmed**



PROBLEM 1: PEAK INDEX IN A MOUNTAIN ARRAY

Problem Statement (Simple)

Ek **mountain array** diya hai:

- pehle strictly increasing
- phir ek peak
- phir strictly decreasing

 Task:

 **peak element ka index return karo**

Example

`arr = [0, 2, 4, 3, 1]`

`index: 0 1 2 3 4`

`Peak = 4 (index 2)`

FIRST-THOUGHT PRINCIPLE

Peak ko directly dhoondhne ki koshish mat karo.

Sirf **direction** pe focus karo:

- upar ja rahe ho? → peak aage
- neeche aa rahe ho? → peak peeche

 Direction detect karna = Binary Search

DECISION RULE AT MID

Compare:

`arr[mid] vs arr[mid + 1]`

Case 1: $arr[mid] < arr[mid + 1]$

- Increasing slope
- Peak **RIGHT** side me

→ $start = mid + 1$

Case 2: $arr[mid] > arr[mid + 1]$

- Decreasing slope
- Peak **LEFT** ya **MID** me

→ $end = mid$

CODE (CLEAN + GUARANTEED)

```
int peakIndexInMountainArray(vector<int>& arr) {

    int start = 0;

    int end = arr.size() - 1;

    // Jab tak ek hi candidate na bache
    while (start < end) {

        int mid = start + (end - start) / 2;

        // Agar slope upar ja rahi hai
        if (arr[mid] < arr[mid + 1]) {

            start = mid + 1;    // left half discard

        }

        // Agar slope neeche aa rahi hai
```

```
        else {  
            end = mid;           // right half discard  
        }  
    }  
  
    // start == end → peak index  
    return start;  
}
```

DRY RUN (STEP-BY-STEP)

`arr = [0, 2, 4, 3, 1]`

`start = 0, end = 4`

`mid = 2`

`arr[2] < arr[3]? → 4 < 3 ❌`

`→ end = 2`

`start = 0, end = 2`

`mid = 1`

`arr[1] < arr[2]? → 2 < 4 ✅`

`→ start = 2`

`start == end == 2`

`Answer = 2`

🕒 TIME & SPACE COMPLEXITY

Time Complexity

- Har step me search space **half** ho rahi hai
👉 $O(\log n)$

Space Complexity

- Sirf variables (`start`, `end`, `mid`)
👉 $O(1)$



PROBLEM 2: SQUARE ROOT OF X

(Binary Search on ANSWER SPACE — LeetCode 69)



Problem Statement

Ek non-negative integer `x` diya hai.
Tumhe $\sqrt{x}$ ka **floor value** nikalna hai.



FIRST-THOUGHT PRINCIPLE

Ye maths problem nahi, **search problem** hai.

Search space:

$1 \rightarrow x$

Condition:

$mid^2 \leq x$

👉 Jaise `mid` badhega, square bhi badhega

👉 **Monotonic behaviour**



ENGINEERING DEPTH: OVERFLOW ISSUE

```
mid * mid
```

Overflow kar sakta hai ❌



SAFE TRANSFORMATION

```
mid ≤ x / mid
```

- ✓ same logic
 - ✓ zero overflow
 - ✓ industry standard
-



CODE :

```
int mySqrt(int x) {  
  
    // Edge cases  
    if (x <= 1) return x;  
  
    int start = 1;  
    int end = x;  
    int ans = 0; // last valid candidate  
  
    while (start <= end) {  
  
        int mid = start + (end - start) / 2;  
  
        // Perfect square  
        if (mid == x / mid) {  
            return mid;  
        }  
    }  
}
```

```

    }

    // mid^2 < x → valid candidate

    else if (mid < x / mid) {

        ans = mid;

        start = mid + 1;

    }

    // mid^2 > x → too large

    else {

        end = mid - 1;

    }

}

// Floor sqrt

return ans;

}

```



DRY RUN (x = 8)

start = 1, end = 8

mid = 4

$4 \leq 8/4$? $\rightarrow 4 \leq 2$ ❌

$\rightarrow$ end = 3

start = 1, end = 3

mid = 2

$2 \leq 8/2$? $\rightarrow 2 \leq 4$ ✅

ans = 2

start = 3

start = 3, end = 3

mid = 3

$3 \leq 8/3 ? \rightarrow 3 \leq 2$ ❌

$\rightarrow$ end = 2

Loop ends $\rightarrow$ return ans = 2

🕒 TIME & SPACE COMPLEXITY

Time

- Range har step me half
👉 $O(\log x)$

Space

- Constant variables only
👉 $O(1)$
-

🌀 PROBLEM 3: FIND MINIMUM IN ROTATED SORTED ARRAY

(Binary Search on PARTIALLY SORTED STRUCTURE — LeetCode 153)

🔗 Problem Statement

Ek sorted array ko kisi unknown pivot par rotate kiya gaya hai (elements **distinct** hain).

🎯 Task:

👉 minimum element find karo in $O(\log n)$

Example

`nums = [4, 5, 6, 7, 0, 1, 2]`

`Minimum = 0`

FIRST-THOUGHT PRINCIPLE

Rotation ke baad bhi:

- ek half **hamesha sorted** hota hai
- minimum **unsorted break** ke paas hota hai

👉 Tum poora array check kar rahe ho = galat approach

MASTER DECISION RULE

Compare:

`nums[mid] vs nums[end]`

Case 1: `nums[mid] > nums[end]`

- mid left sorted block me
- minimum **RIGHT** side me

→ `start = mid + 1`

Case 2: `nums[mid] ≤ nums[end]`

- mid right sorted block me
- minimum **LEFT ya MID** me

→ `end = mid`

CODE (PRODUCTION READY)

```
int findMin(vector<int>& nums) {  
  
    int start = 0;  
    int end = nums.size() - 1;  
  
    while (start < end) {  
  
        int mid = start + (end - start) / 2;  
  
        // Minimum right half me hai  
        if (nums[mid] > nums[end]) {  
            start = mid + 1;  
        }  
        // Minimum left ya mid me hai  
        else {  
            end = mid;  
        }  
    }  
  
    // start == end → minimum element  
    return nums[start];  
}
```

DRY RUN

```
nums = [4, 5, 6, 7, 0, 1, 2]
```

```
start = 0, end = 6
```

```
mid = 3 → nums[mid]=7, nums[end]=2
```

```
7 > 2 
```

```
→ start = 4
```

```
start = 4, end = 6
```

```
mid = 5 → nums[mid]=1, nums[end]=2
```

```
1 <= 2 
```

```
→ end = 5
```

```
start = 4, end = 5
```

```
mid = 4 → nums[mid]=0, nums[end]=1
```

```
0 <= 1 
```

```
→ end = 4
```

```
start == end == 4
```

```
Answer = 0
```

TIME & SPACE COMPLEXITY

Time

- Har iteration me half array discard
👉 $O(\log n)$

Space

- Constant memory

👉 $O(1)$

🧠 LECTURE 15 — FINAL MENTAL MODEL

Binary Search ke 3 FINAL FORMS:

- 1 **On Structure** → peak, rotation
- 2 **On Answer Space** → sqrt, optimization
- 3 **On Sorted Data** → classic search

📌 Core same:

Decision at mid + guaranteed elimination

🌟 FINAL LOCK-IN SUMMARY

Problem	Pattern	Decision
Peak Index	Structure	Slope
sqrt(x)	Answer Space	$mid \leq x/mid$
Rotated Min	Structure	mid vs end
Time		$O(\log n)$
Space		$O(1)$

🔥 FINAL GYAAN :

“ Coding ka matlab correct answer dhoondhna nahi hai.
Coding ka matlab **wrong answers ko confidently eliminate** karna hai. ”